ROB Internal Logic

Overview

The Reorder Buffer (ROB) absorbs out-of-order read completions from MC Core and emits them in-order per AXID to Merge Logic. Depth is tied to `MC_RD_BUF = 64`. Since `RESP_READY` must always be held high, the ROB can never stall incoming completions — backpressure is instead applied by halting new read requests when occupancy reaches the watermark threshold.

Input Signals

Signal	Width	Source	Description
`alloc_req`	1	Tag Allocator	Pulse: allocate a new ROB slot
`AXID[5:0]`	6	Tag Allocator	AXI ID of transaction being allocated
`seqnum[1:0]`	2	Tag Allocator	Seqnum of transaction being allocated
`tag[7:0]`	8	Tag Allocator	Full tag (used to write Lookup Table)
`RESP_VALID`	1	MC Core	Completion valid strobe
`resp_type[1:0]`	2	MC Core	00=RD_DATA, 01=WR_ACK, 10=ERR; ROB acts only on RD_DATA
`tag[7:0]`	8	MC Core	Completion tag; `[7:2]`=AXID, `[1:0]`=seqnum
`data[511:0]`	512	MC Core	Read data payload
`status[3:0]`	4	MC Core	Completion status/error code

Table 1: ROB input signals

Output Signals

Signal	Width	Destination	Description
`data[511:0]`	512	Merge Logic	In-order read data beat
`status[3:0]`	4	Merge Logic	Status for emitted beat
`AXID[5:0]`	6	Merge Logic	AXID of emitted beat
`rob_last`	1	Merge Logic	Asserted on final beat of a transaction; sourced from RTT, passed through
`PKT_READY`	1	Packet FIFO	Deasserted when watermark hit; halts new read requests
`RESP_READY`	1	MC Core	Tied high permanently

Table 2: ROB output signals

Blocks

Free List

- 64-bit bitmask. `1 = free`, `0 = occupied`.
- Allocate: priority encoder selects lowest free bit, clears it, outputs `free_slot_idx[5:0]`.
- Free: Emit Logic sets the bit at `slot_idx` via `free_req`.
- Outputs `free_slot_idx` to Lookup Table (write), ROB BRAM (init), and Head Pointer Array (first allocation).

Occupancy Counter

- Up/down counter, range 0–64.
- Incremented by `alloc_req` from Free List.
- Decremented by `free_req` from Emit Logic.
- Asserts `watermark_hit` when count $\geq$ `MC_RD_BUF` $-$ `ROB_WATERMARK` $= 38$.

Lookup Table

- 256-entry register file, addressed by {`AXID[5:0], seqnum[1:0]`} (8-bit address).
- Each entry stores `slot_idx[5:0]`.
- Write port: Tag Allocator writes `slot_idx` at {`AXID, seqnum`} on allocation.
- Read port: Completion Input reads `slot_idx` using {`tag[7:2], tag[1:0]`}.
- Clear: Emit Logic clears entry at {`AXID, seqnum`} after emit.

ROB BRAM

- Dual-port BRAM, 64 $\times$ 517 bits per slot: `valid[0], data[511:0], status[3:0]`.
- Write port: Completion Input writes `data`, `status`, `valid=1` at `slot_idx`.
- Write port (init): Free List writes `valid=0` at `free_slot_idx` on allocation.
- Read port: Head Pointer Array supplies `head[AXID]` as read address; all 64 head slots checked in parallel each cycle.
- 1-cycle read latency — Emit Logic must account for this in its valid check timing.

Completion Input

- Receives `RESP_VALID, resp_type[1:0], tag[7:0], data[511:0], status[3:0]` from MC Core.
- **Filters on `resp_type`:** acts only when `resp_type == 2'b00` (RD_DATA). Completions with `resp_type == 2'b01` (WR_ACK) are destined for the Response Tracker and are ignored by the ROB. ERR completions (`resp_type == 2'b10`) for read transactions are written into the ROB with the error status preserved so DECERR can be forwarded to the R Response Generator.
- Extracts `AXID = tag[7:2]`, `seqnum = tag[1:0]`.
- Reads `slot_idx` from Lookup Table using {`AXID, seqnum`}.
- Writes `data`, `status`, `valid=1` to ROB BRAM at `slot_idx`.

Head Pointer Array

- 64 registers of 6 bits each, one per AXID. Stores slot index of next expected beat.

- Written on first allocation only: `head[AXID] = free_slot_idx` when the AXID has no in-flight slots (occupancy transitions 0→1 for that AXID). On subsequent allocations for the same AXID, `head[AXID]` is *not* updated — it is owned exclusively by Emit Logic from that point forward.
- Advanced by Emit Logic after each emit: incremented to the next expected slot index.
- Reset to the next `free_slot_idx` only when all in-flight slots for an AXID have been emitted and a new allocation arrives (occupancy returns to 0 then rises again).
- All 64 head pointers drive the ROB BRAM read port simultaneously each cycle.

Emit Logic

- Reads `valid` bits from all 64 head slots out of ROB BRAM each cycle.
- Priority encoder selects lowest AXID whose head slot has `valid=1`.
- On selection: reads `data` and `status` from BRAM; emits to Merge Logic along with `AXID` and `rob_last`. `rob_last` is *not* generated inside the ROB — it is sourced from the Read Transaction Table (RTT), which asserts it when `completed_packets == expected_packets` for that transaction. The ROB passes it through to Merge Logic coincident with the final data beat.
- After emit:
  - Clears `valid` in BRAM slot.
  - Advances `head[AXID]` in Head Pointer Array.
  - Sends `free_req + slot_idx` to Free List (sets bitmask bit).
  - Decrements Occupancy Counter via `free_req`.
  - Clears Lookup Table entry at {`AXID, seqnum`}.
- Maximum one emission per cycle (single R channel path).

PKT_READY Control

- Purely combinational.
- Asserts `PKT_READY=0` to Packet FIFO when `watermark_hit` is high.
- Halts new read requests to MC Core; prevents ROB overflow since `RESP_READY` cannot be deasserted.

Allocation Path

1. Tag Allocator sends `alloc_req` (pulse) with `AXID`, `seqnum`, `tag`.
2. Free List pops `free_slot_idx` (priority encoder on bitmask), clears the bit.
3. Free List writes `slot_idx` to Lookup Table at {`AXID, seqnum`}.
4. Free List initializes ROB BRAM slot: `valid=0` at `free_slot_idx`.
5. Free List stores `free_slot_idx` into `head[AXID]` (first allocation for that AXID only).
6. Occupancy Counter increments.

Completion Path

1. MC Core asserts `RESP_VALID` with `tag[7:0]`, `data[511:0]`, `status[3:0]`.
2. Completion Input extracts `AXID = tag[7:2]`, `seqnum = tag[1:0]`.
3. Completion Input reads `slot_idx` from Lookup Table.
4. Completion Input writes `data`, `status`, `valid=1` to ROB BRAM at `slot_idx`.

5. RESP_READY is tied high — MC Core never stalled on this path.

Emit Path

1. Each cycle, all 64 head[AXID] values drive the ROB BRAM read port.
2. BRAM returns valid bits for all 64 head slots (1-cycle latency).
3. Priority encoder in Emit Logic selects lowest AXID with valid=1.
4. Emit Logic reads data and status from selected slot; forwards to Merge Logic with AXID and last_beat.
5. Slot is freed: valid cleared, head[AXID] advanced, Free List bit set, Occupancy Counter decremented, Lookup Table entry cleared.

Backpressure & Overflow Prevention

Signal	Description
watermark_hit	Asserted when occupancy ≥ 38. Drives PKT_READY control.
PKT_READY=0	Stops Packet FIFO from issuing new read requests to MC Core.
RESP_READY=1	Tied high permanently. ROB can never stall MC Core completions.

Watermark headroom = ROB_WATERMARK = 26. Ensures all requests already accepted by MC Core can return completions without overflowing the ROB.

Key Parameters

Parameter	Value
ROB depth	64 entries (MC_RD_BUF)
Slot width	517 bits (1 valid + 512 data + 4 status)
Free list width	64 bits
Lookup table	256 entries $\times$ 6-bit slot index
Head pointers	64 $\times$ 6-bit registers
Watermark	38 (64 $-$ 26)
Max emissions	1 per cycle

Timing Notes

- ROB BRAM has 1-cycle read latency. Emit Logic valid check must be pipelined accordingly — register the read address, use valid bit one cycle after presenting head[AXID].
- Priority encoder across 64 valid bits is the critical path in Emit Logic. May need pipelining at high frequencies.
- Allocation and completion can occur in the same cycle — write port (completion) and init write (allocation) are independent BRAM ports.
- watermark_hit is combinational from the Occupancy Counter — no additional latency before PKT_READY deasserts.
- Simultaneous alloc_req and free_req on the Free List bitmask in the same cycle is safe — they always target different bits (alloc clears the lowest free bit; free sets the bit of the just-emitted slot, which is by definition occupied). No read-modify-write conflict arises.